

Sandbox de partículas interactivo 2D

Comparación de ejecución en CPU secuencial vs ejecución en CUDA

Vicente Zapata González

ROL: 202073573-5

(ILI257) Programación Paralela Aplicada

Profesor: Jorge Díaz Matte

Jueves 25 de junio de 2026

- Se implementó una simulación 2D con interfaz gráfica, donde el usuario puede observar y modificar partículas en tiempo real.
- Cada partícula posee posición, velocidad y material.
- Las partículas reaccionan al colisionar según reglas personalizadas.
- El objetivo principal es mostrar cómo una simulación costosa se beneficia del paralelismo en GPU.

Problema computacional

En cada frame, cada partícula se compara contra todas las demás:

$$\text{comparaciones por frame} = N \times N$$

- Con $N = 1024$: aproximadamente 1 millón de comparaciones.
- Con $N = 8192$: aproximadamente 67 millones de comparaciones.
- Con $N = 16384$: aproximadamente 268 millones de comparaciones.

Aunque CPU y CUDA mantienen complejidad $O(N^2)$, la GPU puede repartir el trabajo entre miles de hebras.

Versión CPU

- C++ secuencial.
- Doble buffer de partículas.
- Un ciclo externo por partícula.
- Un ciclo interno contra todas las demás.
- Render con OpenGL.

Versión CUDA

- Una hebra por partícula.
- Doble buffer en GPU.
- Kernel tiled $O(N^2)$.
- Uso de memoria compartida `__shared__`.
- Render con OpenGL.

Nota: la estructura del proyecto reutiliza como base las ideas trabajadas en los laboratorios del curso: host C++/CUDA, medición, buffers y render con OpenGL.

CPU secuencial

La versión CPU calcula cada partícula una por una:

```
for (int i = 0; i < count; ++i) {  
    Particle p = in[i];  
  
    for (int j = 0; j < count; ++j) {  
        if (i == j) continue;  
        aplicar_interaccion_cpu(p, in[j]);  
    }  
  
    integrar_y_rebotar_cpu(p, dt);  
    out[i] = p;  
}
```

- Es simple y directa.
- No usa hebras ni paralelismo explícito.
- Aumentar N impacta muy fuerte en FPS.

CUDA tiled

En CUDA, cada hebra procesa una partícula:

```
__shared__ Particle tile[Config::LOCAL_SIZE];

for (int tileStart = 0; tileStart < N; tileStart += blockDim.x) {
    tile[threadIdx.x] = inParticles[tileStart + threadIdx.x];
    __syncthreads();

    for (int k = 0; k < tileCount; ++k) {
        aplicar_interaccion_cuda(p, tile[k]);
    }

    __syncthreads();
}
```

- Se reutilizan bloques de partículas en memoria compartida.
- Cada hebra escribe solamente su propia partícula.

Interacción de materiales

Se definieron tres materiales con comportamiento distinto:

- **Verde:** material liviano, acelera con mayor facilidad.
- **Azul:** material normal, recibe una transferencia suave de velocidad.
- **Rojo:** material pesado, responde menos al choque.

Además:

- Ninguna partícula puede quedar con velocidad cero.
- Se limita la velocidad máxima para mantener una simulación legible.
- El usuario puede seleccionar color con 1, 2, 3 y emitir partículas con click.

- Tiempo por frame medido con `std::chrono`.
- Primeros 30 frames descartados como calentamiento.
- FPS promedio actualizado en consola cada 60 frames.
- Misma cantidad N y misma lógica $O(N^2)$ en CPU y CUDA.

Equipo usado:

- CPU: Intel Core i5-11400F, 6 núcleos / 12 hebras.
- GPU: NVIDIA GeForce RTX 4060, 3072 núcleos CUDA.

Ejecución del sandbox durante la presentación
Comparación directa: CPU secuencial vs CUDA
Cantidad de partículas ajustada según el equipo disponible

Resultados

N	CPU FPS	CUDA FPS	Speedup
1024	383	600	1.57x
2048	147	552	3.76x
4096	46	530	11.64x
8192	12	369	29.93x
16384	3	338	110.37x

N corresponde a la cantidad total de partículas simuladas.

- Al aumentar N , el costo cuadrático se vuelve dominante en CPU.
- En $N = 16384$, CPU baja a aproximadamente 3 FPS.
- CUDA mantiene más de 300 FPS incluso con 268 millones de comparaciones por frame.
- El speedup crece desde 1.57x hasta 110.37x.

Esto muestra que la GPU no cambia la complejidad teórica, pero sí cambia drásticamente la capacidad práctica de procesar el trabajo.

Dificultades encontradas

- OpenCL en WSL no expuso correctamente la GPU NVIDIA.
- Se decidió usar CUDA como backend GPU principal.
- Fue necesario mantener reglas equivalentes en CPU y CUDA.
- Se debió evitar que varias hebras escribieran la misma partícula.
- Hubo que ajustar velocidades para evitar simulaciones visualmente caóticas.

Cumplimiento de la propuesta

La propuesta indicaba que una simulación de partículas se beneficiaría de una implementación paralela porque:

- muchas partículas deben actualizar posición, velocidad y estado;
- la comparación directa entre partículas escala muy rápido;
- muchas operaciones pueden calcularse de manera independiente;
- la GPU puede procesar múltiples partículas simultáneamente.

Los resultados confirman esa hipótesis: al crecer N , la CPU secuencial cae fuertemente, mientras CUDA mantiene una simulación fluida.

- Se implementó un sandbox 2D interactivo con reglas personalizadas.
- Se comparó CPU secuencial contra CUDA usando la misma lógica $O(N^2)$.
- CUDA obtuvo mejoras crecientes al aumentar el número de partículas.
- La mayor mejora medida fue de aproximadamente 110x con $N = 16384$.
- El proyecto cumple el objetivo de demostrar visual y cuantitativamente el beneficio del paralelismo en GPU.

Gracias por su atención